SDR-Based Doppler Radar

Velocity measurement and speed camera with PlutoSDR and GNU Radio, Arduino display and buzzer alarm

S/LT Bilal PN, S/LT Sanaullah PN, S/LT Farhan PN, S/LT Huzaifa PN
Advisor: Assistant Professor Mohammad Mustaqim

1. Objective

- Build a low-cost, software-defined CW Doppler radar in **GNU Radio** that measures target speed in real time.
- Make the carrier **frequency** and the **maximum speed** adjustable live from the GUI.
- Work like a street speed camera: hold each vehicle's peak speed and **capture** vehicles above a speed limit.
- Give feedback on a 4-digit LED display and a buzzer whose loudness, pitch and beep rate follow the target.

2. Principle

$$fd = 2 \cdot v \cdot fc / c \quad \rightarrow \quad v = fd \cdot c / (2 \cdot fc)$$

A wave reflected from a moving target is shifted by the Doppler frequency fd. At fc = 2.482 GHz, 1 m/s gives 16.5 Hz; a car at 50 km/h gives about 230 Hz. Positive fd = approaching, negative = receding.

3. System architecture

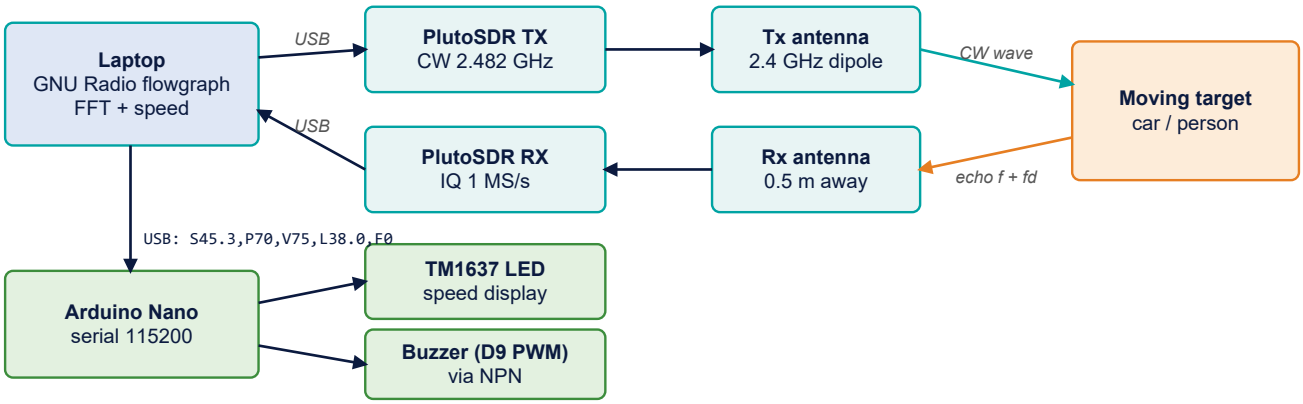


Figure 1 - One PlutoSDR transmits and receives at the same time; the laptop does all signal processing; the Arduino drives the display and buzzer.

4. Hardware

Component	Role
ADALM-PLUTO SDR (AD9363)	Full-duplex transceiver: CW transmitter and IQ receiver, USB 2.0
2 × 2.4 GHz dipole antennas + SMA cable	Separate Tx and Rx antennas, about 0.5 m apart
Laptop (Windows, GNU Radio 3.10 / radioconda)	Flowgraph: down-conversion, FFT, detection, display, logging
Arduino Nano + TM1637 4-digit LED	Live speed, then the held speed: blinks 5 s, keeps blinking if captured
Passive buzzer + NPN (2N2222/BC547) + 1 kΩ + diode	Audible alarm driven by Timer1 PWM on pin D9

5. Signal-processing workflow

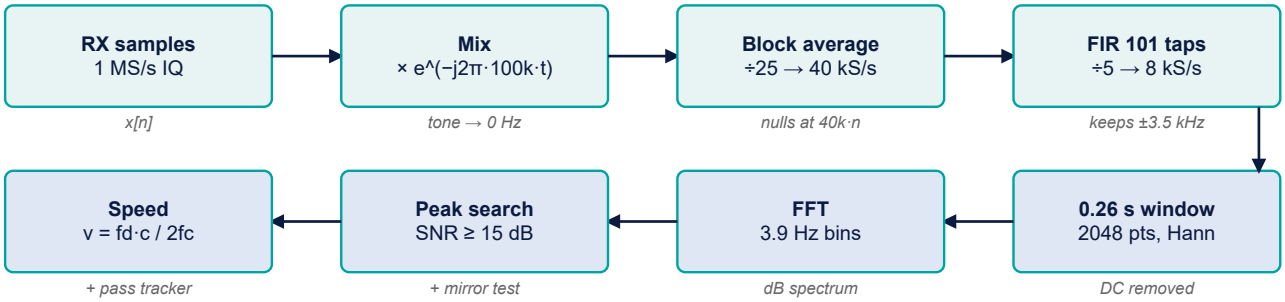


Figure 2 - Processing chain, executed every 64 ms on the laptop.

- **Transmit:** a CW tone at 2.482 GHz + 100 kHz. The offset keeps the echo away from the receiver's DC offset.
- **Down-convert:** mix the 100 kHz tone to 0 Hz and decimate 1 MS/s → 8 kS/s (block average, then FIR).
- **Spectrum:** 2048-point FFT over 0.26 s (Hann window) → 3.9 Hz bins = 0.85 km/h; peak interpolation gives a 0.1 km/h readout. The short window makes the radar respond fast.
- **Detect:** the peak must be 15 dB above the median noise floor and above a learned clutter map, 6 dB above its mirror frequency (rejects fans) and present in 2 consecutive frames.

GNU Radio flowgraph (gnuradio/doppler_radar.grc)

GNU Radio block	Setting	Purpose
Signal Source	100 kHz complex tone, amplitude 0.5	CW waveform (offset from 0 Hz)
PlutoSDR Sink	LO = Frequency slider, 1 MS/s, TX attenuation slider	Transmits the CW tone
PlutoSDR Source	LO = Frequency slider, 1 MS/s, manual RX gain slider	Receives TX leakage + echoes
Frequency Xlating FIR Filter	shift -100 kHz, low-pass, decimate 25	Tone to 0 Hz, 1 MS/s to 40 kS/s
FIR Filter	low-pass 3.5 kHz, decimate 5	40 kS/s to 8 kS/s Doppler signal
Doppler Speed Camera (embedded Python)	Max speed, Speed limit and Min signal sliders	FFT, clutter map, detection, km/h, capture, log, Arduino
QT GUI Frequency Sink / Waterfall	8 kHz span, 4096 / 1024 points	Live Doppler spectrum
Function Probe + QT GUI Label	polled 5 times per second	LIVE SPEED and LAST SPEED readouts
QT GUI Range (6 sliders)	Frequency, Max speed, Speed limit, RX gain, TX attenuation, Min signal	Live adjustment

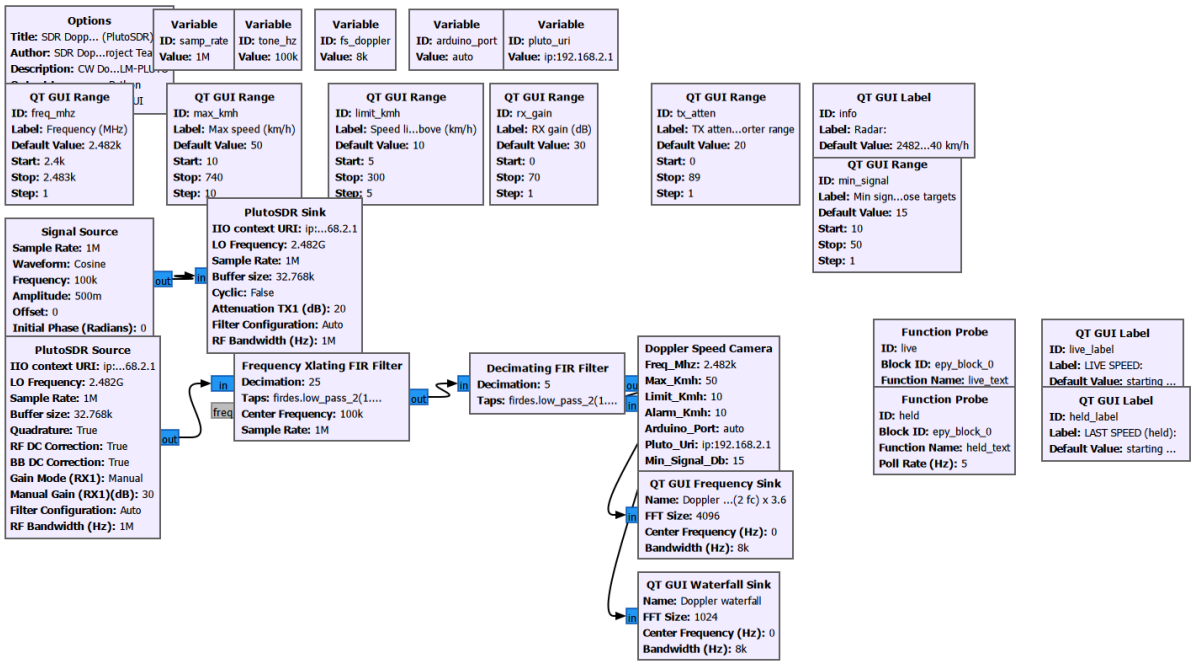


Figure 3 - The flowgraph in GNU Radio Companion (EDIT_GNURADIO.bat): variables and sliders at the top, PlutoSDR Sink/Source, filters, Doppler Speed Camera block, displays and function probes.

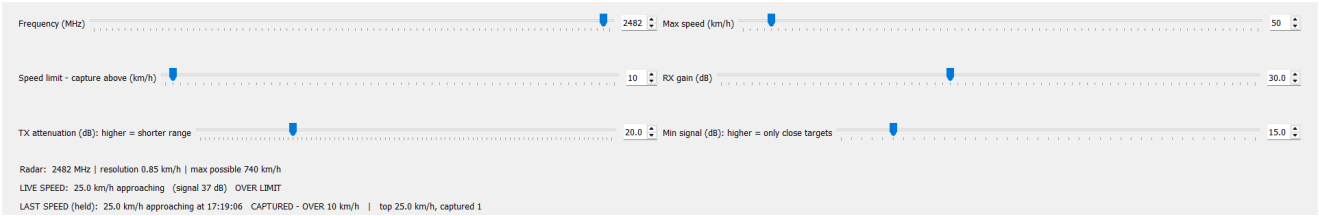


Figure 4 - GNU Radio radar window, controls: six sliders, radar info, LIVE SPEED and LAST SPEED.

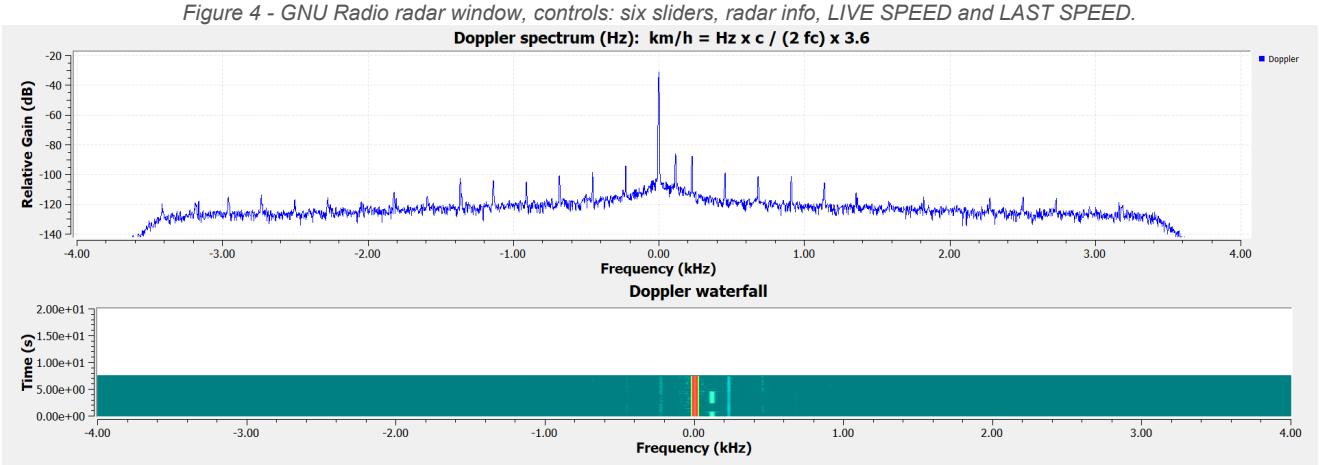


Figure 5 - Doppler spectrum and waterfall with a test target at +115 Hz (25 km/h), captured over the 10 km/h limit.

6. Speed-camera logic

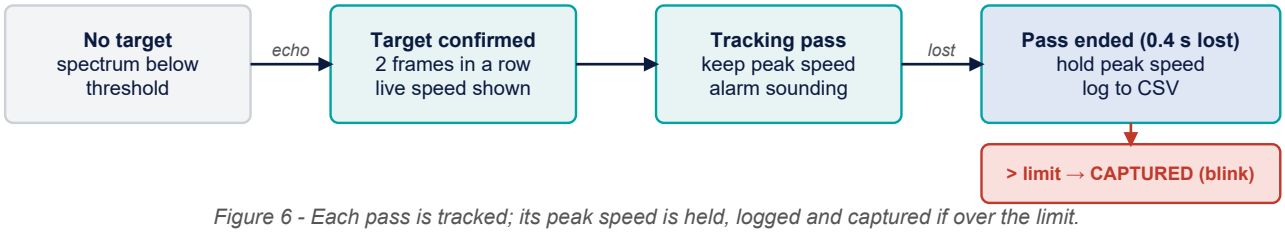


Figure 6 - Each pass is tracked; its peak speed is held, logged and captured if over the limit.

7. Outputs: display and buzzer

Output	Behaviour
Laptop screen	GNU Radio window: sliders, live speed, LAST SPEED, captures, top speed, spectrum, waterfall
LED display	Live speed during a pass; then the peak speed, blinking 5 s (kept blinking if over the speed limit); shown again after a restart; ---- = no pass yet
Buzzer volume	PWM duty cycle 2-50 %: faster target = louder
Buzzer pitch	600-1200 Hz: faster target = higher tone
Beep rate	1-10 beeps/s: stronger echo (closer target) = faster beeps
Speed log	speed_log.csv: date, time, speed, direction, limit, captured

8. Specifications and results

Parameter	Value
Carrier / waveform	2.482 GHz default, adjustable 2400-2483 MHz; continuous wave (ISM band)
Software	GNU Radio 3.10 flowgraph (main), Python version with the same detector (backup)
Sample rate	1 MS/s, decimated to 8 kS/s
Speed resolution	0.85 km/h per FFT bin, 0.1 km/h readout
Response	live speed ~0.15 s after movement starts; held speed ~0.75 s after the target leaves
Speed range	3.3 km/h to the Max speed slider (default 50, up to 740 km/h), both directions
Measured accuracy	Test targets injected through the Pluto, 5-700 km/h: error below 0.05 km/h, correct direction and capture
Room tests	Nobody moving: no false detections. Walking person: signal 24-47 dB, detected in every 5 s of movement. Toy car: detected when driven straight at the antennas at close range
Update rate	64 ms per frame
Safe shutdown	Transmitter muted (-89 dB) on every exit, verified on hardware

9. Development workflow

Stage	Work done	Outcome
1. Design	Doppler theory, carrier choice (2.482 GHz ISM), resolution and range budget	Specification: 0.85 km/h bins, 3.3-50 km/h (up to 740)
2. RF bring-up	Pluto drivers, Python control, TX/RX loop-back test	Peak at +100 kHz; RX gain set to 30 dB
3. Signal processing	Mixer, decimation, FFT, noise floor, mirror test, peak interpolation	Real-time loop at 64 ms per frame
4. Speed camera	Pass tracking, last-speed hold, capture over limit, CSV log	Peak speed held and logged
5. Display and buzzer	Arduino sketch, serial protocol, TM1637 display, Timer1 PWM buzzer	Live and held speed; alarm follows speed and distance
6. GNU Radio flowgraph	Pluto source/sink, filters, displays, sliders, Doppler Speed Camera block	Live frequency and max-speed control
7. Integration and test	One-click launchers, safe TX shutdown, accuracy tests, walking and vehicle tests	Complete system demonstrated

10. Default settings and range

Slider (block ID)	Default	Meaning
Frequency (freq_mhz)	2482 MHz	Carrier; 2400-2483 MHz with the 2.4 GHz antennas
Max speed (max_kmh)	50 km/h	Highest speed measured; up to 740 km/h at 2482 MHz
Speed limit (limit_kmh)	10 km/h	Faster passes are CAPTURED: logged and kept blinking on the LED
TX attenuation (tx_atten)	20 dB	Higher = less power = shorter range (x1.8 range per 10 dB)
Min signal (min_signal)	15 dB	Signal a target needs above the noise; higher = only closer targets
RX gain (rx_gain)	30 dB	As high as possible without the CLIPPING warning

- Sliders act immediately but are not saved. To change a default permanently: **EDIT_GNURADIO.bat** → double-click the slider block (e.g. max_kmh) → Default Value → OK → Ctrl+S → start the radar again.
- **More range** (e.g. a toy car): TX attenuation 15 and Min signal 12; aluminium foil on the car; antennas low and pointed along its path, TX and RX 0.5 m apart. **Less range**: raise TX attenuation or Min signal.
- Doppler radar measures movement towards or away from the antennas: a target crossing sideways or standing still gives no speed.

11. Running the system

- Connect the Pluto (middle USB port) and the Arduino to the laptop.
- Double-click **START_GNURADIO.bat**: the GNU Radio radar window opens with the default settings.
- Stand still for 3 s while it learns the background; set frequency, max speed and limit with the sliders.
- Close the window to stop (TX is muted). **EDIT_GNURADIO.bat** shows the block diagram; **START_RADAR.bat** runs the Python backup.



PlutoSDR with antennas

12. Future work

- FMCW waveform for range as well as speed.
- Camera snapshot of captured vehicles.
- Directional patch antennas for longer range; multi-target tracking.